

PIMPRI CHINCHWAD EDUCATION TRUST'S

PIMPRI CHINCHWAD COLLEGE OF ENGINEERING

NBA Accredited | NAAC Accredited with 'A' Grade | An Autonomous Institute
AICTE Approved | Permanently Affiliated to SPPU, Pune

Department of Information Technology

Academic Year: 2026-2027

Assignment No. 8: Optimizing Delivery Routes for a Logistics Company (TSP - Least Cost Branch & Bound)

Name	:	**Om Rajendra Bharambe**
PRN	:	124B1F001
Branch	:	Information Technology (IT)
Semester	:	V
Batch	:	A1
Course Code	:	BIT25PC04
Course Name	:	**Design & Analysis of Algorithms Laboratory**
Guided By	:	Dr. Rajesh N. Phursule
GitHub Repository	:	om29dev/DAA_Lab_Submission
Assignment Link	:	Assignment 8 on GitHub

Assignment No. 8: Optimizing Delivery Routes for a Logistics Company (TSP - Least Cost Branch & Bound)

Problem Statement

A leading logistics company, SwiftShip, is responsible for delivering packages to multiple cities. To minimize fuel costs and delivery time, the company needs to find the shortest possible route that allows a delivery truck to visit each city exactly once and return to the starting point. The company wants an optimized solution that guarantees the least cost route, considering:

- Varying distances between cities.
- Fuel consumption costs, which depend on road conditions.
- Time constraints, as deliveries must be completed within a given period.

Since there are N cities, a brute-force approach checking all (N-1)! permutations is infeasible for large N (e.g., 20+ cities). Therefore, you must implement an LC (Least Cost) Branch and Bound algorithm to find the optimal route while reducing unnecessary computations efficiently.

Course Objective

1. To know the basics of computational complexity of various algorithms.

2. To select appropriate algorithm design strategies to solve real-world problems.

Course Outcomes

1. Apply branch & Bound technique to solve problems

2. Classify P, NP, NP-complete, NP-Hard problems

Theory

1. The problem is the Travelling Salesman Problem (TSP): find minimum-cost cycle visiting each city once.

2. Branch & Bound explores partial solutions (paths) as nodes in a search tree. Each node has:

   a. a partial path,

   b. current cost so far,

   c. a lower bound estimate on completion cost (computed via matrix reduction).

3. We expand nodes in increasing lower-bound order (best-first). If a node's lower bound >= current best solution cost, prune it.

4. Matrix reduction: set impossible edges to infinity, then for each row subtract row minimum, for each column subtract column minimum. The sum of subtractions is a lower bound contribution.

5. LC Branch & Bound with reduced matrix is exact and often prunes massively compared to brute-force.

Algorithm (LC Branch & Bound - Summary)

Build initial cost matrix of size N x N (cost[i][j] = fuel-cost from i -> j; diagonal = infinity). Reduce matrix - compute initial lower bound lb0.

Create root node with reduced matrix, lb=lb0, path = [startCity], level = 0. Use a priority queue (min-heap) keyed by node.lb. Pop the node with smallest lb. If node.level == N-1 (one city left), finalize path and update best tour + cost. Else for each feasible city j not in path:

1. Create child: copy parent's matrix, set row of parent city to infinity, set column j to infinity, set [j][start] to infinity when closing final state accordingly, set edge parent -> j as used (cost already accounted).

2. Reduce child's matrix to compute new lower bound: child.lb = parent.lb + cost[parent -> j] + reductionCost.

3. If child.lb < bestCost, push child to PQ; else prune.

Continue until PQ empty.

Best solution at termination is optimal route.

Pseudo code

```
procedure TSP_BranchAndBound(costMatrix, start):
    rootMatrix, lb0 = reduceMatrix(costMatrix)
    root = Node(matrix=rootMatrix, lb=lb0, path=[start], level=0)
    bestCost = +INF
    bestPath = null
    PQ = min-heap ordered by lb
    PQ.push(root)

    while PQ not empty:
        node = PQ.pop()
        if node.lb >= bestCost: continue // prune

        if node.level == N-1:
            // complete tour: add last city -> start cost
            last = node.path.back()
```

```
            totalCost = node.lb + originalCost[last][start]
            if totalCost < bestCost:
                bestCost = totalCost; bestPath = node.path + [start]
            continue

        u = node.path.back()
        for each v not in node.path:
            childMatrix = copy(node.matrix)
            // block row u and column v and set [v][u] = INF (prevent
immediately returning)
            setRowToInf(childMatrix, u)
            setColToInf(childMatrix, v)
            childCost = node.lb + originalCost[u][v]
            reductionCost = reduceMatrixInPlace(childMatrix)
            child.lb = childCost + reductionCost
            child.path = node.path + [v]
            child.level = node.level + 1
            if child.lb < bestCost: PQ.push(child)

    return bestCost, bestPath
```

Complexity & Notes

1. Worst-case time still factorial, but bounding + reduced cost often prunes large parts of tree.

2. Memory: PQ can grow large; used memory depends on pruning effectiveness.

3. Practical for exact solution up to ~12-16 cities comfortably; higher N may be possible with strong pruning or problem structure.

4. Handling time constraints: one can modify bounding to add penalties for late arrival or disallow paths that exceed time windows.

Complexity Analysis

1. Best case: pruning is so powerful we examine very few nodes.

2. Worst case: behaves like factorial time O(N!) (no pruning possible).

3. Bounding (matrix-reduction) cost per node: O(N^2) to copy & reduce matrix.

4. Practical observation: Branch & Bound can solve N ≈ 10-15 exactly within seconds; for larger N, runtime grows quickly.

Example Input and Output

Delivery Cost Matrix (4 x 4 Cities):

From / To	City 0	City 1	City 2	City 3
City 0	0	10	15	20
City 1	10	0	35	25
City 2	15	35	0	30
City 3	20	25	30	0

Optimization Result:

Parameter	Optimized Output Value
Optimal Tour Route	City 0 -> City 1 -> City 3 -> City 2 -> City 0
Minimum Delivery Cost	80 units
Cost Breakdown	0->1 (10) + 1->3 (25) + 3->2 (30) + 2->0 (15) = 80
Status	Global Optimal Minimum Cost Found

Conclusion

LC Branch & Bound with reduced cost matrix gives an exact TSP algorithm that drastically reduces the search space compared to naive permutation enumeration by using strong lower bounds and best-first expansion. It is suitable for SwiftShip when the number of stops per route is moderate and exact optimality is required (e.g., high-cost deliveries or regulatory constraints). For larger route sets or real-time routing, combine exact BnB for critical sub routes with heuristics (Christofides, 2-opt, or metaheuristics) for scalability.

Code and Output

Repository: https://github.com/om29dev/DAA_Lab_Submission

Assignment 8 Folder: https://github.com/om29dev/DAA_Lab_Submission/tree/main/Assignment 8

10.1 C++ Implementation Source Code (swiftship_delivery.cpp):

```
#include <bits/stdc++.h>
using namespace std;

const int INF = 1e9;

struct Node {
    int cost, bound, city, level;
    vector<int> path;

    bool operator>(const Node& other) const {
        return bound > other.bound;
    }
};

int n;
vector<vector<int>> costMatrix;
int bestCost = INF;
vector<int> bestPath;

int calculateBound(Node node) {
    int bound = node.cost;

    for (int i = 0; i < n; i++) {
        if (find(node.path.begin(), node.path.end(), i) != node.path.end())
            continue;

        int minimum = INF;

        for (int j = 0; j < n; j++) {
            if (i != j &&
                (find(node.path.begin(), node.path.end(), j) ==
node.path.end() || j == 0)) {
                minimum = min(minimum, costMatrix[i][j]);
            }
        }

        if (minimum != INF)
            bound += minimum;
```

```
    }

    if (node.level == n)
        bound = node.cost + costMatrix[node.city][0];

    return bound;
}

void leastCostBranchAndBound() {
    priority_queue<Node, vector<Node>, greater<Node>> pq;

    Node root;
    root.cost = 0;
    root.city = 0;
    root.level = 1;
    root.path = {0};
    root.bound = calculateBound(root);

    pq.push(root);

    while (!pq.empty()) {
        Node current = pq.top();
        pq.pop();

        if (current.bound >= bestCost)
            continue;

        if (current.level == n) {
            int totalCost = current.cost + costMatrix[current.city][0];

            if (totalCost < bestCost) {
                bestCost = totalCost;
                bestPath = current.path;
                bestPath.push_back(0);
            }

            continue;
        }

        for (int next = 1; next < n; next++) {
            if (find(current.path.begin(), current.path.end(), next)
                != current.path.end())
                continue;
```

```
            Node child;
            child.path = current.path;
            child.path.push_back(next);
            child.city = next;
            child.level = current.level + 1;
            child.cost = current.cost + costMatrix[current.city][next];
            child.bound = calculateBound(child);

            if (child.bound < bestCost)
                pq.push(child);
        }
    }
}

int main() {
    cout << "SwiftShip Delivery Route Optimization\n";
    cout << "-------------------------------------\n";

    cout << "Enter the number of cities: ";
    cin >> n;

    if (n < 2) {
        cout << "Please enter at least 2 cities.\n";
        return 0;
    }

    costMatrix.assign(n, vector<int>(n));

    cout << "\nEnter the cost matrix (" << n << " x " << n << "):\n";
    cout << "Enter 0 for the distance from a city to itself.\n";

    for (int i = 0; i < n; i++) {
        cout << "Enter costs from City " << i << ": ";

        for (int j = 0; j < n; j++) {
            cin >> costMatrix[i][j];
        }
    }

    leastCostBranchAndBound();

    cout << "\n----------- Optimal Route -----------\n";

    if (bestCost == INF) {
```

```
        cout << "No valid route found.\n";
        return 0;
    }

    cout << "Minimum Delivery Cost: " << bestCost << '\n';
    cout << "Optimal Route: ";

    for (int i = 0; i < bestPath.size(); i++) {
        cout << "City " << bestPath[i];

        if (i != bestPath.size() - 1)
            cout << " -> ";
    }

    cout << "\n--------------------------------------\n";

    return 0;
}
```

10.2 Execution Output:

Compile and run swiftship_delivery.cpp in the terminal environment:

```
om29dev@DESKTOP-MH6NVIC:~/tsp$ vi swiftship_delivery.cpp
om29dev@DESKTOP-MH6NVIC:~/tsp$ g++ swiftship_delivery.cpp -o swiftship_delivery
om29dev@DESKTOP-MH6NVIC:~/tsp$ ./swiftship_delivery

SwiftShip Delivery Route Optimization
-------------------------------------
Enter the number of cities: 4

Enter the cost matrix (4 x 4):
Enter 0 for the distance from a city to itself.
Enter costs from City 0: 0 10 15 20
Enter costs from City 1: 10 0 35 25
Enter costs from City 2: 15 35 0 30
Enter costs from City 3: 20 25 30 0

---------- Optimal Route ------------
Minimum Delivery Cost: 80
Optimal Route: City 0 -> City 1 -> City 3 -> City 2 -> City 0
--------------------------------------
om29dev@DESKTOP-MH6NVIC:~/tsp$
```

Figure 8.1: SwiftShip Delivery Route Optimization Execution Output